WebRTC Meet

Detailed Technical & Functional Documentation

A browser-based video meeting application using WebRTC, FastAPI, WebSocket signaling, ICE/STUN/TURN, recording, scheduling, and offline speech transcription.

Item	Implementation
Project type	Real-time browser video meeting application
Backend	Python FastAPI + Uvicorn
Frontend	HTML, CSS and JavaScript
Real-time signaling	WebSocket
Media transport	WebRTC peer-to-peer
ICE infrastructure	STUN/TURN relay configured through an external Metered relay service
Recording	Browser MediaRecorder + server chunk upload
Transcription	faster-whisper running locally on the server

Security note: the repository contains TURN credentials in the client-side JavaScript. This documentation intentionally does not reproduce those credentials. In a production deployment, TURN credentials should be rotated and moved to a secure credential-generation/configuration mechanism.

Prepared from the supplied project source code

1. Project Overview

WebRTC Meet is a lightweight meeting application that allows users to create an instant meeting, share a meeting link, join an existing meeting, schedule a meeting, control microphone/camera, record the call, and generate live captions and a downloadable transcript.

The important architectural idea is that the FastAPI server does not normally carry the actual camera and microphone media. Instead, browsers create WebRTC peer connections. The FastAPI WebSocket endpoint acts as the signaling channel used to exchange the information required to establish those connections.

1.1 Main objectives

- Create a unique meeting room with a short room code.
- Allow multiple participants to join the same room using one URL.
- Exchange WebRTC SDP offers/answers and ICE candidates through WebSocket signaling.
- Use ICE servers so browsers can discover a viable network path even when users are behind NAT/firewalls.
- Use an external TURN relay service when a direct peer-to-peer route is not possible.
- Provide microphone and camera controls.
- Record the meeting from one participant's browser and upload the recording in chunks.
- Run speech-to-text locally with faster-whisper and broadcast captions to all participants.
- Schedule meetings in memory and generate reusable meeting links.

1.2 Technology stack

Layer	Technology	Purpose
Frontend	HTML/CSS/JavaScript	UI and browser-side WebRTC logic
Backend	FastAPI	REST APIs, WebSocket signaling and static file serving
Server	Uvicorn	ASGI server running FastAPI
Real-time signaling	WebSocket	Peer discovery and WebRTC signaling message relay
Media	WebRTC	Audio/video communication between browsers
ICE	STUN + TURN	NAT traversal and relay fallback
TURN provider	Metered relay service	Externally hosted STUN/TURN infrastructure configured in ICE_SERVERS
Recording	MediaRecorder + Canvas + Web Audio	Composite recording in browser
Speech-to-text	faster-whisper + PyAV	Offline/local transcription
Persistence	Local filesystem + memory	Recordings/transcripts on disk; rooms/schedules in RAM

2. Project Structure

```
webrtc-meet-work/
├── main.py
├── requirements.txt
├── .env
├── README.md
├── static/
│   ├── index.html
│   ├── room.html
│   ├── style.css
│   ├── home.js
│   ├── call.js
│   └── features.js
└── data/
    ├── recordings/
    ├── transcripts/
    └── models/
```

2.1 Backend: main.py

main.py is the central server application. It creates the FastAPI app, initializes the data directories, loads faster-whisper in the background, exposes REST endpoints, handles WebSocket connections, maintains in-memory room state, processes recording chunks, and serves the frontend.

2.2 Frontend files

- index.html: landing page for starting, joining and scheduling meetings.
- home.js: home-page API calls, local name storage, room-link generation and scheduled-meeting list operations.
- room.html: pre-join screen, call controls, video grid, recording/transcript panel.
- call.js: WebRTC, WebSocket signaling, peer management, camera/mic controls and leaving the room.
- features.js: recording, live transcription, captions, transcript download and recording list handling.
- style.css: visual styling.

2.3 Data folders

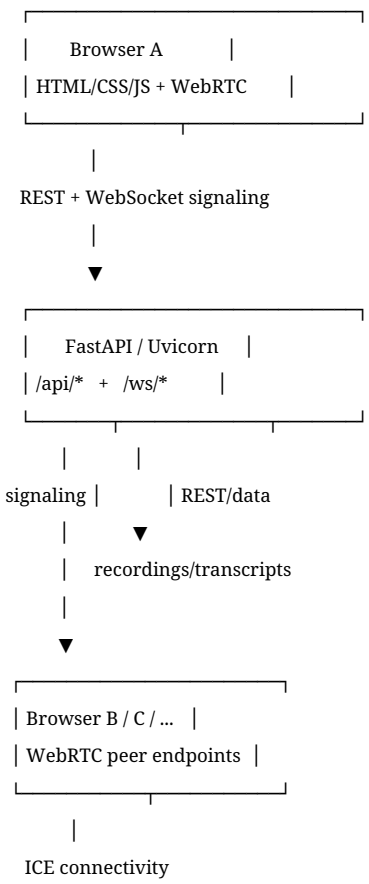
- data/recordings: generated WebM/MP4 meeting recordings.
- data/transcripts: one JSONL transcript file per room.
- data/models: local faster-whisper model files.

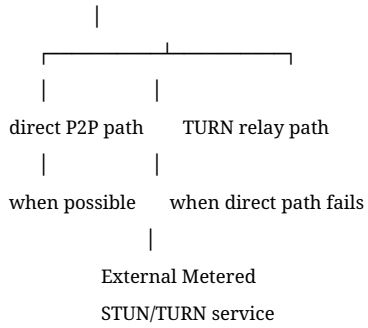
2.4 Dependencies

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
websockets==13.1
pydantic==2.9.2
faster-whisper==1.2.1
```

3. Complete System Architecture

The application has four important paths: application/API traffic, WebSocket signaling, WebRTC media traffic, and recording/transcription traffic.





3.1 What the FastAPI server does

- Creates room IDs.
- Maintains the list of connected WebSocket clients per room.
- Relays WebRTC signaling messages from one browser to another.
- Tracks room-wide recording/transcription status.
- Receives recording chunks and stores them.
- Serves finished recordings.
- Receives microphone clips for transcription.
- Runs Whisper locally and broadcasts transcript segments.
- Creates and lists scheduled meeting records.

3.2 What the browser does

- Requests camera and microphone access.
- Creates RTCPeerConnection objects.
- Adds local audio/video tracks to every peer connection.
- Generates and accepts SDP offers/answers.
- Generates ICE candidates.
- Uses the configured STUN/TURN servers for ICE.
- Renders remote streams as video tiles.
- Records the visible call composition when recording is enabled.

4. WebRTC and ICE/STUN/TURN Explanation

WebRTC is responsible for the actual real-time audio/video connection. Before media can flow, two browsers must discover how they can reach each other. This process is called ICE (Interactive Connectivity Establishment).

4.1 STUN

STUN helps a browser discover its public-facing network address as seen from outside the local network. If both participants can establish a suitable direct route, the media can travel directly between browsers.

4.2 TURN

TURN is a relay mechanism. When a direct connection cannot be established because of restrictive NAT, firewall rules, or network topology, WebRTC can send media through a TURN server. TURN is therefore an important reliability component for real-world deployments.

4.3 ICE configuration used by this project

The supplied call.js defines an ICE_SERVERS array containing one STUN URL and several TURN URLs. The TURN entries include UDP-style access, TCP transport and TLS/TURN over port 443 variants. The project uses an externally hosted Metered relay service rather than implementing a TURN server inside main.py.

```
const ICE_SERVERS = [
 { urls: 'stun:stun.relay.metered.ca:80' },
 {
  urls: 'turn:global.relay.metered.ca:80',
  username: '<TURN_USERNAME>',
```

```
    credential: '<TURN_CREDENTIAL>'
   },
   {
    urls: 'turn:global.relay.metered.ca:80?transport=tcp',
    username: '<TURN_USERNAME>',
    credential: '<TURN_CREDENTIAL>'
   },
   {
    urls: 'turn:global.relay.metered.ca:443',
    username: '<TURN_USERNAME>',
    credential: '<TURN_CREDENTIAL>'
   },
   {
    urls: 'turns:global.relay.metered.ca:443?transport=tcp',
    username: '<TURN_USERNAME>',
    credential: '<TURN_CREDENTIAL>'
   }
  ];
```

The actual credentials from the supplied project are deliberately replaced above with placeholders. They should not be committed to a public repository because client-side JavaScript can expose static TURN credentials.

4.4 Why multiple TURN URLs are configured

- Port 80 gives a commonly reachable TURN endpoint.
- TCP transport provides an alternative when UDP is restricted.
- Port 443 is useful on networks that permit HTTPS-like traffic more readily.
- turns: enables TLS-protected TURN transport.
- ICE can test available candidates and select a working path.

5. How to Configure a TURN/COTURN Server

There are two approaches. The supplied project uses an external Metered STUN/TURN relay. If the project is moved to an organization-controlled infrastructure, a self-hosted Coturn server can replace that external relay.

5.1 Current project approach: external TURN

1. Create an account with a TURN provider such as Metered.
2. Create/obtain STUN/TURN credentials from the provider dashboard.
3. Copy the provider's ICE server configuration.
4. Place the ICE server entries in the WebRTC client configuration.
5. Use HTTPS for the meeting application when accessed from other devices.
6. Test from different networks, especially a mobile hotspot and a corporate/restricted network.

5.2 Self-hosted Coturn alternative

Coturn is an open-source STUN/TURN server. A typical Linux deployment uses a public VPS with a public IP address. The application server and Coturn server may be the same machine or separate machines, but separating them is often easier to manage.

```
# Ubuntu/Debian example
sudo apt update
sudo apt install coturn


# Enable the service according to the distribution's coturn configuration.
# A typical /etc/turnserver.conf contains settings similar to:


listening-port=3478
```

```
tls-listening-port=5349
fingerprint
lt-cred-mech
realm=meet.example.com
server-name=meet.example.com

# Use a strong authentication secret or managed user credentials.
# If TLS is enabled, configure valid certificate/key paths.
```

The exact Coturn configuration depends on the hosting provider, public/private IP topology, TLS certificates and authentication model. Firewall rules must allow the TURN listening ports and the configured relay port range.

5.3 Client configuration after Coturn deployment

```
const ICE_SERVERS = [
 { urls: "stun:turn.example.com:3478" },
 {
  urls: "turn:turn.example.com:3478",
  username: TURN_USERNAME,
  credential: TURN_PASSWORD
 },
 {
  urls: "turns:turn.example.com:5349",
  username: TURN_USERNAME,
  credential: TURN_PASSWORD
 }
];
```

A production system should not hard-code long-lived static credentials into JavaScript. Prefer short-lived TURN credentials generated by a backend endpoint or another secure mechanism supported by the TURN provider.

5.4 Important distinction

The supplied repository does NOT contain a Coturn server implementation. It contains the WebRTC client ICE configuration that consumes a remote STUN/TURN service. This distinction should be stated clearly in a project viva or technical presentation.

6. Home Page Flow and API Calls

6.1 Start an instant meeting

When the user clicks Start instant meeting, home.js sends a GET request to /api/new-room.

```
const res = await fetch('/api/new-room');
const data = await res.json();
location.href =
 `/room.html?room=${data.room}&name=${encodeURIComponent(name)}`;
```

The backend creates an 8-character hexadecimal room ID using uuid.uuid4().hex[:8].

```
@app.get("/api/new-room")
def new_room():
   return {"room": uuid.uuid4().hex[:8]}
```

6.2 Join an existing meeting

Joining does not require a separate backend call. The browser extracts the room code from either a raw code or a full meeting URL and navigates directly to room.html.

6.3 Schedule a meeting

```
POST /api/schedule
Content-Type: application/json
```

```
{
 "title": "Project Meeting",
 "time": "2026-10-05T15:00",
 "host_name": "Host"
}
```

The backend generates a new room ID and stores the meeting in the scheduled_meetings list.

6.4 List scheduled meetings

```
GET /api/schedule
```

The response is sorted by the stored meeting time. The frontend creates Copy link, Join and Delete buttons for each entry.

6.5 Delete a scheduled meeting

```
DELETE /api/schedule/{meeting_id}
```

The backend removes the matching meeting from the in-memory list. There is no database in the supplied implementation, so scheduled meetings disappear when the server restarts.

6.6 API summary

Method	Endpoint	Purpose
GET	/api/new-room	Generate instant meeting room ID
POST	/api/schedule	Create scheduled meeting
GET	/api/schedule	List scheduled meetings
DELETE	/api/schedule/{meeting_id}	Delete scheduled meeting

7. WebSocket Signaling and Peer Connection Flow

The WebSocket endpoint is the central signaling mechanism:

```
WS /ws/{room_id}/{client_id}
```

The server accepts the connection, reports existing peers, reports current room feature status, stores the WebSocket, and informs existing participants that a new peer joined.

7.1 Join sequence

7. Browser obtains camera and microphone with getUserMedia().
8. Browser creates a random clientId with crypto.randomUUID().
9. Browser opens WebSocket /ws/{room}/{clientId}.
10. Server responds with existing-peers.
11. Server responds with current recording/transcription status.
12. Browser sends hello with the participant's display name.
13. For each existing peer, the newcomer creates a peer connection as initiator.
14. The initiator creates an SDP offer and sends it through WebSocket.
15. The receiver sets the remote description, creates an SDP answer and sends it back.
16. Both sides exchange ICE candidates.
17. Once ICE and SDP negotiation succeeds, media begins flowing.

7.2 Signaling messages

Message type	Sender	Receiver	Purpose
hello	Browser	Server	Provides display name; server does not need to process it further.
existing-peers	Server	New browser	Lists peers already in the room.
new-peer	Server	Existing browsers	Notifies existing participants about a newcomer.
offer	Initiator	Target browser	Carries SDP offer.
answer	Receiver	Initiator	Carries SDP answer.
ice-candidate	Either browser	Target browser	Carries ICE candidate.
peer-left	Server	Remaining browsers	Removes disconnected participant.
get-status	Browser	Server	Requests current room feature status.
status	Server	Browsers	Synchronizes recording/caption state.
set-recording	Browser	Room	Starts/stops recording state.

set-transcription	Browser	Room	Starts/stops captions/transcription state.

7.3 Offer/answer logic

```
const pc = new RTCPeerConnection({ iceServers: ICE_SERVERS });

localStream.getTracks().forEach(track =>
  pc.addTrack(track, localStream)
);

const offer = await pc.createOffer();
await pc.setLocalDescription(offer);

ws.send({
  type: "offer",
  target: peerId,
  sdp: pc.localDescription
});
```

The server does not create SDP and does not handle media. It only forwards the signaling message to the target WebSocket.

8. Peer Management, Video and User Controls

8.1 Peer data structures

```
const peers = {};            // peerId -> RTCPeerConnection
const pendingCandidates = {};  // peerId -> queued ICE candidates
const peerNames = {};        // peerId -> display name
```

Each remote participant gets its own RTCPeerConnection. For a small group, this creates a mesh topology. With N participants, each browser maintains connections to the other participants.

8.2 Local media

```
localStream = await navigator.mediaDevices.getUserMedia({
  video: true,
  audio: true
});
```

The browser asks the user for permission. The local stream is displayed immediately and its tracks are added to each peer connection.

8.3 Receiving remote media

```
pc.ontrack = (event) => {
  addTile(peerId, event.streams[0], 'Participant', false);
};
```

Every received stream becomes a video tile in the video grid.

8.4 Microphone toggle

```
localStream.getAudioTracks().forEach(t => t.enabled = micOn);
```

This does not renegotiate the WebRTC connection. It simply enables or disables the local audio track.

8.5 Camera toggle

```
localStream.getVideoTracks().forEach(t => t.enabled = camOn);
```

8.6 Leave flow

18. Finish any active recording upload.
19. Close every peer connection.
20. Stop local media tracks.
21. Close the WebSocket.

22. Navigate back to the home page.
23. The server detects WebSocketDisconnect and broadcasts peer-left to remaining users.

8.7 Group topology

The implementation is a peer-to-peer mesh, not an SFU architecture. This keeps the system simple, but bandwidth and CPU requirements increase as participant count grows. The project's README describes it as suitable for small groups rather than large webinars.

9. Meeting Recording System

Recording is implemented primarily in the browser. The browser that starts recording collects the visible participant video tiles and the audio from the streams, creates a composite MediaStream, and records it using MediaRecorder.

9.1 Start recording API

```
POST /api/recordings?room=<room_id>&ext=webm
```

The server generates a recording ID, creates an empty file under data/recordings, and returns the ID.

9.2 Browser composition

- A 1280×720 canvas is created.
- All video elements in #videoGrid are inspected.
- Video frames are drawn onto the canvas in a grid.
- Audio tracks are connected to an AudioContext MediaStreamDestination.
- The canvas capture stream supplies the video track.
- The combined audio/video stream is passed to MediaRecorder.

9.3 Chunk upload

```
PUT /api/recordings/{rec_id}/chunks
Body: binary MediaRecorder chunk
```

The browser produces approximately five-second MediaRecorder chunks. Each chunk is appended to the server-side recording file. A small retry loop is used if an upload fails.

9.4 Complete recording

```
POST /api/recordings/{rec_id}/complete
```

The server finalizes the recording. It can remux the file with ffmpeg when available, or use PyAV. The objective is to produce a file with usable duration and seeking metadata.

9.5 List and download recordings

Method	Endpoint	Purpose
GET	/api/rooms/{room}/recordings	List recording files and statuses
GET	/api/recordings/{rec_id}/download	Download a finished recording

9.6 Important recording behavior

Only the browser that started the recording performs the composite capture. Therefore, that browser should remain open and preferably visible until recording is saved. If the recording owner disconnects, the server starts finalization for the active recording.

10. Live Captions and Offline Transcription

The project includes a local speech-to-text pipeline based on faster-whisper. No paid transcription API is required. The server downloads the selected model on first use and stores it under data/models.

10.1 Whisper startup

```
WHISPER_MODEL = os.getenv("WHISPER_MODEL", "small")
WHISPER_COMPUTE = os.getenv("WHISPER_COMPUTE", "int8")
WHISPER_DEVICE = os.getenv("WHISPER_DEVICE", "cpu")
```

The model is loaded in a background thread during FastAPI startup. The /api/whisper-status endpoint exposes loading, ready or error state.

10.2 Transcription activation

When the CC button is enabled, the room sends set-transcription through WebSocket. Every participant then records their own microphone in short clips. Because each browser submits its own audio together with its name, the server can associate the recognized text with the correct speaker.

10.3 Transcription API

```
POST /api/transcribe?room=<room_id>&name=<speaker_name>&dur_ms=8000
Content-Type: audio/webm or another supported browser audio type

Body: binary microphone clip
```

The server first checks whether transcription is enabled and whether the Whisper model is ready. It decodes the audio to 16 kHz mono float32 using PyAV, checks for silence, and then calls faster-whisper.

10.4 Transcript storage

```
data/transcripts/<room>.jsonl
```

Each recognized segment is stored as one JSON object per line. The server also broadcasts the segment through WebSocket as a transcript message.

10.5 Transcript retrieval

Method	Endpoint	Purpose
GET	/api/whisper-status	Check speech-model state
POST	/api/transcribe	Transcribe one browser microphone clip
GET	/api/rooms/{room}/transcript	Return transcript JSON
GET	/api/rooms/{room}/transcript?format=text	Return transcript as text
GET	/api/rooms/{room}/transcript?format=text&download=1	Download transcript .txt

10.6 Transcription limitations

- There can be noticeable delay because the browser sends approximately 8-second clips.
- CPU inference can add additional delay.
- The project limits pending clips to avoid an unlimited queue.
- Silent or nearly silent clips are skipped.
- Speaker attribution is based on the browser's submitted participant name.

11. Complete REST API Reference

This section consolidates every REST route implemented in main.py.

#	Method	Endpoint	Input	Output / Function
1	GET	/api/new-room	None	JSON room code
2	POST	/api/schedule	title, time, host_name	Created meeting object
3	GET	/api/schedule	None	Sorted meeting list
4	DELETE	/api/schedule/{meeting_id}	Path ID	{ok:true}
5	POST	/api/recordings	room, ext	Recording ID
6	PUT	/api/recordings/{rec_id}/chunks	Binary body	Uploaded byte count
7	POST	/api/recordings/{rec_id}/complete	Path ID	Final recording status
8	GET	/api/rooms/{room}/recordings	Room ID	Recording list
9	GET	/api/recordings/{rec_id}/download	Recording ID	Binary file
10	GET	/api/whisper-status	None	Whisper state
11	POST	/api/transcribe	room, name, dur_ms + binary audio	Transcript segment or skip/error
12	GET	/api/rooms/{room}/transcript	format, download	JSON or text transcript

11.1 Example schedule request

```
fetch('/api/schedule', {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({
```

```
    title: 'Team Meeting',
    time: '2026-10-05T15:00',
    host_name: 'Host'
  })
});
```

11.2 Example room creation response

```
{
  "room": "a1b2c3d4"
}
```

11.3 Example recording response

```
{
  "id": "f0e1d2c3b4a5"
}
```

11.4 Example transcript segment

```
{
  "name": "Alice",
  "text": "Good morning everyone",
  "time": 1790840000.12
}
```

12. Complete WebSocket API Reference

12.1 Connection

```
wss://<host>/ws/<room_id>/<client_id>
```

When the site is served over HTTPS, call.js automatically uses wss://. For local HTTP development, it uses ws://.

12.2 Client → Server messages

Type	Important fields	Meaning
hello	name	Participant introduction
offer	target, sdp	Send SDP offer to peer
answer	target, sdp	Send SDP answer to peer
ice-candidate	target, candidate	Send ICE candidate
set-recording	active, recordingId	Change recording state
set-transcription	active	Change caption/transcription state
get-status	None	Request room status

12.3 Server → Client messages

Type	Important fields	Meaning
existing-peers	peers[]	Peers already connected
new-peer	peer	New participant notification
offer	sender, sdp	Relayed SDP offer
answer	sender, sdp	Relayed SDP answer
ice-candidate	sender, candidate	Relayed ICE candidate
peer-left	peer	Participant disconnected
status	recording, recordingId, transcription	Shared feature state
transcript	segment	Recognized speech segment

12.4 Candidate queue

ICE candidates can arrive before the remote SDP description is set. The client therefore stores candidates in pendingCandidates[peerId]. After setRemoteDescription succeeds, flushPendingCandidates() adds the queued candidates to the RTCPeerConnection.

12.5 Server relay behavior

The server adds sender = client_id to signaling messages and forwards them to the target WebSocket. This keeps the backend simple: it does not understand the SDP contents; it acts as a signaling relay.

13. Deployment, HTTPS and Environment Configuration

13.1 Local run

```
python -m venv .venv
# activate the environment
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8000
```

The application can be opened at http://localhost:8000 for local testing. Camera and microphone access is generally permitted for localhost.

13.2 Public deployment

For other users on the internet, the application should be deployed behind HTTPS. A public reverse proxy such as Nginx can terminate TLS and forward normal HTTP requests and WebSocket connections to Uvicorn.

```
Browser
  | HTTPS / WSS
  ▼
Nginx / Load Balancer
  | HTTP / WS
  ▼
Uvicorn
  |
FastAPI
```

13.3 Environment variables

Variable	Default	Purpose
HOST	0.0.0.0	Server bind address
PORT	8000	Server port
DATA_DIR	./data	Data storage root
WHISPER_MODEL	small	Speech model size/path
WHISPER_COMPUTE	int8	Whisper compute type
WHISPER_DEVICE	cpu	cpu or cuda
WHISPER_CPU_THREADS	0	CPU thread configuration
WHISPER_LANGUAGE	auto	Optional language code such as en/hi
MAX_PENDING_CLIPS	4	Transcription back-pressure limit
FFMPEG_PATH	auto/none	Optional ffmpeg executable path

13.4 HTTPS requirement

For non-localhost users, browser camera/microphone APIs require a secure origin. Therefore the public deployment should use https://. The WebSocket automatically becomes wss:// because call.js checks location.protocol.

13.5 TURN deployment checklist

- Confirm the TURN service is reachable from the public internet.
- Use valid TURN credentials.
- Test both UDP and TCP connectivity.
- Keep a TURN/TLS 443 option for restrictive networks.
- Rotate exposed credentials if they have ever been committed publicly.
- Do not expose TURN credentials unnecessarily in logs or documentation.

14. Security, Limitations and Improvements

14.1 Security observations from the current code

- There is no user authentication or authorization.
- A person who knows a room ID can attempt to join it.
- Recording and transcript endpoints do not implement login-based access control.
- TURN credentials are present in the browser source in the supplied implementation.
- Scheduled meetings are stored only in memory.

- There is no database-backed audit trail.
- Room state is held in process memory, so a multi-instance deployment would require shared state.

14.2 Recommended production improvements

24. Move TURN credentials to a secure backend-generated credential flow and rotate the credentials currently present in source.
25. Add user authentication and authorization.
26. Protect recording and transcript download endpoints.
27. Use a database such as PostgreSQL for scheduled meetings and metadata.
28. Use Redis or another shared coordination layer if multiple application instances are deployed.
29. Add rate limiting for room creation, transcription and recording endpoints.
30. Validate room membership before allowing recording/transcript access.
31. Add structured logging and monitoring.
32. Use an SFU architecture for larger meetings instead of full mesh.
33. Add automatic cleanup policies for old recordings.

14.3 Scalability limitation

The current WebRTC mesh means every participant sends media independently to other participants. This is simple and effective for small meetings, but it does not scale like an SFU. For larger groups, an SFU such as mediasoup, Janus, LiveKit or another WebRTC media server architecture would reduce each browser's upload burden.

14.4 Recording limitation

Recording is performed by one browser rather than by a dedicated media server. The recording participant therefore becomes an important part of the recording pipeline. A production media-server recording system would be more resilient.

15. End-to-End Call Flow — Simple Explanation

This is the easiest way to explain the entire project during a demo or viva.

34. The user opens the home page and enters a name.
35. The user clicks Start instant meeting.
36. The frontend calls /api/new-room.
37. FastAPI generates a unique room code.
38. The browser opens room.html using that room code.
39. The user clicks Join call and grants camera/microphone permission.
40. The browser creates a WebSocket connection to /ws/{room}/{clientId}.
41. The server tells the browser which participants already exist.
42. The browser creates RTCPeerConnection objects for the required peers.
43. The browser exchanges SDP offers and answers through the FastAPI WebSocket.
44. The browsers exchange ICE candidates through the same WebSocket.
45. ICE uses STUN to discover connectivity and TURN as a relay fallback.
46. Once negotiation succeeds, audio/video flows through WebRTC.
47. The FastAPI server continues to carry signaling and feature-control messages, not the normal media stream.
48. If the user turns on recording, the browser composes the meeting into a MediaRecorder stream.
49. The recording is uploaded to FastAPI in chunks and stored under data/recordings.
50. If CC is enabled, each browser records its own microphone in short clips.
51. FastAPI sends the clips to faster-whisper locally and receives recognized text.
52. The transcript is stored in data/transcripts and broadcast to everyone in the room.
53. When a participant leaves, the WebSocket disconnect is detected and peer-left is broadcast.

15.1 One-line architecture summary

Browser UI → FastAPI REST/WebSocket → WebRTC signaling → ICE/STUN/TURN connectivity → browser-to-browser media, with FastAPI separately handling recording uploads, local transcription and meeting scheduling.

16. Viva / Presentation Explanation

16.1 What did you build?

I built a browser-based WebRTC meeting application. Users can create or join rooms through a shareable link. The frontend is written in HTML, CSS and JavaScript, while FastAPI handles REST APIs and WebSocket signaling.

16.2 Why WebRTC?

WebRTC provides real-time audio and video directly between browsers. This avoids sending normal call media through the application server and gives low-latency communication.

16.3 Why is a signaling server required?

WebRTC still needs a way for peers to exchange SDP offers, answers and ICE candidates. I used FastAPI WebSocket as the signaling server. It forwards these messages to the correct participant.

16.4 Why STUN and TURN?

STUN helps browsers discover a reachable network address for direct connectivity. TURN is a fallback relay when direct connectivity fails. In this project I configured an external Metered STUN/TURN relay service in the WebRTC ICE server list.

16.5 Did you build your own Coturn server?

No. The supplied project consumes an externally hosted TURN service. Coturn can be self-hosted as an alternative, but there is no Coturn server implementation in this repository.

16.6 How does recording work?

The recording browser takes the video from the meeting tiles, draws them onto a canvas, mixes audio using Web Audio, records the combined stream with MediaRecorder, and uploads approximately five-second chunks to FastAPI.

16.7 How does live transcription work?

Each participant's browser sends short microphone clips to the server. FastAPI decodes the audio, faster-whisper transcribes it locally, the result is stored as JSONL, and a WebSocket transcript message sends the text to everyone.

16.8 What are the main limitations?

- Mesh WebRTC is designed for small groups.
- There is no login/authentication in the current version.
- Scheduled meetings are in memory.
- Recording depends on the browser that started it.
- Static TURN credentials should be replaced with a secure credential mechanism.
- A production deployment should protect recordings and transcripts.

Appendix A — Function-to-Function Call Map

Function / Component	Calls / Uses	Purpose
home.js → startBtn handler	GET /api/new-room	Create instant room
home.js → scheduleBtn handler	POST /api/schedule	Create scheduled meeting
home.js → loadMeetings()	GET /api/schedule	Load scheduled meetings
home.js → del button	DELETE /api/schedule/{id}	Delete meeting
call.js → joinCall()	getUserMedia(), connectSignaling()	Start local media and signaling
call.js → connectSignaling()	WS /ws/{room}/{client}	Open signaling channel
call.js → createPeerConnection()	RTCPeerConnection(ICE_SERVERS)	Create WebRTC peer
createPeerConnection()	createOffer(), addTrack(), onicecandidate	Initiate negotiation
WebSocket offer handler	setRemoteDescription(), createAnswer()	Answer remote peer
WebSocket ICE handler	addIceCandidate()	Complete ICE negotiation
features.js → recording	POST /api/recordings	Create recording
CompositeRecorder.upload()	PUT /api/recordings/{id}/chunks	Upload recording chunk
CompositeRecorder.stop()	POST /api/recordings/{id}/complete	Finalize recording
features.js → transcription	GET /api/whisper-status	Check model
features.js → transcription	POST /api/transcribe	Send microphone clip
features.js → loadRecordings()	GET /api/rooms/{room}/recordings	Show recordings
features.js → transcript download	GET /api/rooms/{room}/transcript	Download transcript

Appendix B — Files Generated/Used at Runtime

Path	Description
data/recordings/<room>__<id>.webm/mp4	Meeting recording
data/transcripts/<room>.jsonl	Speaker-labeled transcript segments
data/models/	Local faster-whisper model storage

Appendix C — Final Project Statement

The project demonstrates the complete lifecycle of a small browser-based real-time meeting system: room creation, signaling, peer discovery, WebRTC negotiation, ICE connectivity using STUN/TURN, peer media exchange, recording, local speech recognition, transcript distribution, and meeting scheduling. The architecture intentionally keeps the signaling/backend layer simple while allowing the browser to perform real-time media processing.

For production, the next engineering priorities are authentication, secure TURN credential management, protected recording/transcript access, persistent scheduling storage, observability, and an SFU-based media architecture for larger meetings.